A Dimensional Data Warehouse for Geospatial Monitoring of Municipal Public Works, with an Evolution Path Toward a Lakehouse Architecture

Uriel González Casiano[0009-0009-8172-9584], Marco Tulio Maldonado Mejía[0009-0005-4691-9607], and Gabriel Hurtado Avilés*[0009-0002-5686-1822]

Escuela Superior de Cómputo (ESCOM), Instituto Politécnico Nacional, Mexico City, Mexico
{ugonzalezc2400,mmaldonadom2200}@alumno.ipn.mx
ghurtadoa@ipn.mx
gonzalezcasianouriel@gmail.com, marcotuliomm.s45@gmail.com,
gabrielhuav@gmail.com

Abstract. The accelerated growth of public infrastructure data in municipal governments generates heterogeneous volumes that are difficult to manage with isolated spreadsheets or transactional systems. This article presents the design and prototype implementation of a dimensional Data Warehouse for the geospatial monitoring of municipal public works, complemented by object storage (Cloudflare R2) for unstructured evidence —photographic reports and PDF documents— linked bidirectionally to the warehouse. The warehouse adopts a star schema with ten dimensions (five Slowly Changing Dimension Type 2, three Type 1, two Type 0) and two fact tables, and exposes OLAP queries and six analytical views for delay detection, budget alerts, and citizen participation through a single REST API, released as reference code rather than as a hosted service. A published geospatial viewer offers graph-like exploration across works, communities, budgets and citizen proposals. Rather than claiming a full enterprise lakehouse, we deliberately scope the contribution as a warehouse-centric design and outline its evolution toward a lakehouse (an open table format —Iceberg/Parquet— queried directly over the object store) as future work. The prototype was exercised over a seeded synthetic dataset of 1,247 public works from 55 communities and evaluated through a documented, multi-seed protocol for the analytical detection module. Against anomalies generated as latent process states rather than as the detector's own predicates, the rule reaches 0.46 precision at 0.35 recall and does not dominate an unsupervised baseline. The design enables transparent, auditable, and citizen-oriented oversight of public works without the infrastructure costs of enterprise platforms.

Keywords: Data Warehouse · Dimensional Modeling · Slowly Changing Dimensions · Geospatial Monitoring · Open Government Data · Public Works · Object Storage · Lakehouse Architecture

1 Introduction

Transparency in the management of municipal public works is one of the most persistent challenges of digital governance in Mexico. Public-works departments simultaneously

* Corresponding author: gabrielhuav@gmail.com

manage contractors, budgets, field supervisors, progress photographs, monthly reports, and hand-over processes; yet most municipalities operate with distributed file systems, spreadsheets, or, at best, isolated transactional systems that preclude historical analysis, early delay detection, and informed citizen participation.

The *lakehouse* paradigm [1] offers a reference response to this fragmentation: it unifies the flexibility of Data Lake object storage (unstructured data, direct ingestion, low cost) with the analytical capability of a Data Warehouse (star schema, slowly changing dimensions, OLAP views). This work does not implement a full enterprise lakehouse: given the data volume at municipal scale, we build a *dimensional Data Warehouse* as the analytical core and use object storage only as a repository of unstructured evidence linked to it. Full lakehouse convergence —an open table format queried directly over the lake— is proposed as an evolution path (Section 8).

This article presents a prototype for the geospatial monitoring of public works in Temascaltepec, State of Mexico (64,844 inhabitants across 55 communities).

Research Questions

RQ1. Can a dimensional Data Warehouse deployed on commodity managed services (PostgreSQL/Supabase plus S3-compatible object storage) provide auditable historical traceability of municipal public works at the scale of a rural Mexican municipality, without the enterprise platforms normally associated with lakehouse architectures?

RQ2. Which dimensional design decisions —fact grain, choice of Slowly Changing Dimension type per entity, and view materialisation strategy— are required so that delay detection, budget alerting and citizen participation can all be served from a single analytical schema?

RQ3. How effective is a threshold rule expressed over the warehouse's analytical views at recovering anomalies in public-works execution, compared with an unsupervised baseline, and which anomaly classes does it fail to recover?

The three structure both the design and the evaluation: RQ1 is addressed by the architecture and warehouse design (Sections 3 and 4) and by the cost and deployment discussion (Section 8); RQ2 by the grain, SCD and view-strategy analysis (Section 4); and RQ3 by the evaluation of the detection module (Section 7.4).

Contributions

The contribution is threefold: (1) a reference architecture centered on a dimensional *Data Warehouse* with slowly changing dimensions, complemented with S3-compatible object storage for evidence and exposed through a REST API (RQ1); (2) a geospatial map acting as a graph-like exploration interface over works, communities, budgets and proposals (RQ2); and (3) a non-circular evaluation of the detection module against latent causes, with a failure analysis by class (RQ3).

Over a seeded test scenario, the prototype organizes 1,247 simulated public works with an accumulated budget of $127.4 million pesos (Section 7 details the full population). Every figure in this article comes from synthetic data: the warehouse population from a seeded generator, the detection results from ten seeds and four prevalences. They characterise the behaviour of the system, not the municipality of Temascaltepec, and no claim about real public-works execution there is made or implied.

The remainder of the article is organized as follows: Section 2 reviews related work and contrasts the proposal with comparable systems; Section 3 describes the architecture; Section 4 details the warehouse; Section 5 describes the object storage layer; Section 6 presents the geospatial map; Section 7 reports results; and Section 8 concludes.

2 Related Work

2.1 Data Lakehouse and Architecture Convergence

Zaharia et al. [1] coined the term *lakehouse* to describe an architecture that combines the open, flexible storage of a Data Lake with the ACID transaction guarantees, schema control, and analytical performance of a Data Warehouse. The transactional layer that makes this convergence possible was described by Armbrust et al. [9], who show how an open table format over cloud object stores can provide ACID semantics and time travel without a dedicated database engine; this is precisely the mechanism our object layer does not yet implement. Databricks [3] extended this model to the geospatial domain, integrating vector and raster data on a unified platform.

In the present work we adopt the spirit of the lakehouse without requiring Apache Spark or Delta Lake: S3-compatible object storage (Cloudflare R2) acts as the evidence layer and PostgreSQL/Supabase as the warehouse layer, connected by a REST API and by database triggers rather than by an external ETL process. We stress that, in its current state, the object store is a binary-evidence repository and not a queryable analytical lake: no open table format (Iceberg/Delta/Hudi) and no engine querying over the lake are used. We therefore position the system as a dimensional Data Warehouse with object storage, and reserve full lakehouse convergence for future work.

2.2 Geospatial Knowledge Graphs

Knowledge graphs (KGs) have proven useful for integrating heterogeneous government data. Hogan et al. [4] provide a comprehensive survey of the state of the art in KGs, highlighting their application to georeferenced entities. Janowicz et al. [5] present KnowWhereGraph, one of the largest public geospatial graphs, which links human and environmental data under FAIR principles. In the open-government-data domain, Espinoza-Arias et al. [6] document the case of Zaragoza, where a municipal knowledge graph connects public services, infrastructure, and demographic data. Rajabi et al. [8] systematize the construction of knowledge graphs over open government data using Semantic Web technologies.

Our geospatial map adopts an analogous *relational* approach at a rural municipal scale: public works, communities, budgets, and citizen proposals constitute the entities, while reports and photographs represent linked evidence. The distinction with respect to the works above is deliberate and is stated explicitly in Section 6: the graph is implicit in the foreign keys of the dimensional model and is materialised only at the interface level. Publishing the schema as an OWL vocabulary, which would place the system in the same family as [5,6,8], is the first item of our future work.

2.3 Open Government Data and Infrastructure Disclosure Standards

The literature on Open Government Data (OGD) [7] emphasizes that publishing public-infrastructure data in reusable formats increases accountability and facilitates the detection of irregularities. However, most OGD systems in Latin American municipalities are

limited to static download portals, without analytical capability or integration with field data.

A directly relevant body of practice is the family of infrastructure disclosure standards. The Open Contracting for Infrastructure Data Standard (OC4IDS) [10], developed by the Open Contracting Partnership with CoST — the Infrastructure Transparency Initiative, defines a JSON schema for publishing project- and contract-level information across the infrastructure lifecycle. OC4IDS and our proposal are complementary: OC4IDS specifies *what* a government should disclose and in which interchange format, whereas we address *how* a small municipality can produce, historise and analyse that information internally. A natural convergence (Section 8) is to expose the warehouse as an OC4IDS-conformant endpoint.

Four bodies of work bear directly on the design. Spatial OLAP [13] has combined multidimensional analysis with cartographic navigation since the mid-2000s; our map is a SOLAP client in all but name, differing in scope rather than kind, since geometry is a display attribute of `dim_region` and not a hierarchy to drill along. The trade-offs of slowly changing dimensions are surveyed by Faisal and Sarwar [14]; ours is not the mechanism but its application at audit granularity. Ambituuni [15] shows that disclosure alone does not curb irregularity, which is the argument for coupling publication with analysis. And indicator-based detection over procurement records has an empirical tradition [16] —the family our C1–C3 rule belongs to, and against which its modest performance (Section 7.4) should be read. This same approach of consolidating official open data into a queryable analytical artifact has been applied before in other government domains in Mexico, such as seismic risk [11], a methodological antecedent transferred here to the monitoring of public works.

2.4 Positioning with Respect to Comparable Systems

Set against representative systems —the geospatial knowledge graphs, the OC4IDS disclosure standard, the geospatial-lakehouse architecture and the download portal typical of Latin American open-government sites— the proposal differs on two axes and coincides on the rest. Those systems variously offer a formal semantic layer, a richer interchange format or far greater scale; none preserves entity state at audit granularity, and none manages binary field evidence as a first-class citizen linked back to the analytical store.

The semantic-web systems [5,6,8] provide a formal layer we lack, but none answers "what did this record look like the day the budget was modified?", which is what SCD 2 provides; and in OGD portals and OC4IDS implementations documents are referenced by URL without a managed lifecycle. The intersection of *temporal auditability* and *evidence linkage* at municipal scale is where this work sits. That is the position of the proposal, not a claim of vacancy: establishing that no comparable system exists would require a systematic review we have not carried out. This work fills the intersection of *temporal auditability* and *evidence linkage* at a scale where enterprise platforms are not economically viable.

3 System Architecture

Figure 1 illustrates the four layers of the proposed architecture, which follows a layered pattern with a clear separation of responsibilities. It is described as a reference architecture and prototype: the analytical core is the dimensional Data Warehouse, and the

object store operates as an evidence layer. The subsections below correspond one-to-one to the four layers labelled in the figure, so that the terminology of figure and text coincides.

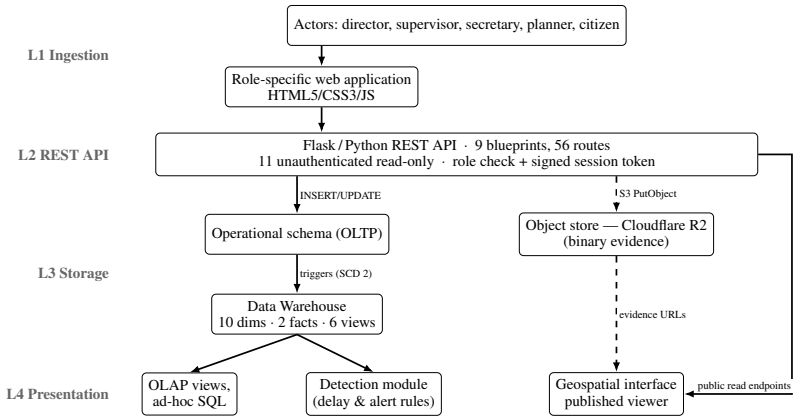

Fig. 1. System architecture (warehouse-centric), redrawn in English. The four layers L1–L4 correspond to Sections 3.1–3.4. Solid arrows: structured-data flow; dashed arrows: unstructured (binary) evidence flow. Released role-specific pages read operational endpoints; the warehouse serves detection and OLAP through `/api/analitica`. The geospatial viewer is part of the released artefact.

3.1 Layer 1: Ingestion

The actors (directors, supervisors, secretaries, planners and registered citizens) interact with a web application built in HTML5/CSS3/JavaScript. Forms send structured data to the API (text, coordinates, amounts) and binary files (JPG/PNG, PDF) to the same endpoint, which routes each to its destination: the relational insert is transactional, while the pairing with object storage is compensated rather than atomic (Section 5).

3.2 Layer 2: REST API

A Flask/Python REST API organised in nine blueprints declares 56 routes, plus a health check on the application. Eleven declarations carry no authentication decorator and expose only `GET`: five under `/api/public/`, two under `/api/analitica/` serving the warehouse views of Section 4.3, and four citizen-participation reads, one of which returns data only against a valid session token. That analytical surface is public by design. Counts are of route declarations, so a route accepting two methods counts once. Access control for staff is a `@require_auth` decorator validating a role identifier carried in request headers; the citizen module uses HMAC-signed session tokens, and passwords are PBKDF2-HMAC-SHA256 hashes with a 16-byte salt. The scheme is deliberately lightweight and is *not* a full token-based authentication service: hardening it —token expiry, rotation, and moving the staff role check out of a header— is listed among the limitations in Section 8.2.

3.3 Layer 3: Storage

Object store (Cloudflare R2). An S3-compatible object store that receives unstructured evidence through direct ingestion. Paths follow a *hive-partitioned* convention, shown in Equation (1):

```
works/{work_id}/reports/{year}-{month}/{ts}_{slug}.{ext}    (1)
```

This structure enables direct exploration by work and period; each object's metadata (public URL, original name, MIME type, size in bytes, upload date) is also persisted in a warehouse-side table, creating a bidirectional link between the two layers.

Data Warehouse (PostgreSQL/Supabase). An analytical layer with a star schema, two fact tables, and ten dimensions (Section 4). The warehouse is populated from the operational schema by database triggers rather than by an external ETL job: eight triggers synchronise the dimensions (applying SCD 2 versioning where required) and four triggers emit rows into the audit fact table when a progress report, a cost record, an image upload or a citizen vote is registered. The warehouse uses range partitioning by year on the audit fact table to optimize historical queries.

3.4 Layer 4: Presentation

The released presentation layer consists of role-specific HTML/CSS/JavaScript pages that consume the REST API. Analytical consumption —the six views of Section 4.3, ad-hoc OLAP queries and the detection module of Section 3.5— is served directly from the warehouse through `/api/analitica`. The geospatial viewer of Figure 3 is published alongside the rest of the artefact and reads a frozen projection of the same synthetic dataset, so map and workspaces never disagree.

3.5 Analytical Detection Module

The module is stateless, reads the warehouse and emits three artefacts: a *delay set* (`v_delayed_works`) of every work whose latest monthly row carries the delay flag with its slippage; an *alert stream* (`v_audit_alerts`) classifying audit events into five categories through a `CASE` expression over the fact table; and an *anomaly set* (`v_anomalies_detection`) applying the C1–C3 criteria below to each work–month row. The third, not the other two, is what Section 7.4 evaluates.

The rule applied to a work–period record flags it as anomalous if *any* of three operational criteria holds:

C1. *Cost outlier*: the executed budget deviates from the mean by more than three standard deviations, $|b_i - \mu_b|/\sigma_b > 3$.
C2. *Extreme delay*: the recorded slippage exceeds 120 days.
C3. *Progress inconsistency*: physical progress below 30% together with financial progress above 80%.

The corresponding anomaly score is $s_i = \max\left(\frac{|b_i-\mu_b|}{\sigma_b},\ \frac{d_i}{120}\mathbb{1}[d_i > 120],\ 3.5 \cdot \mathbb{1}[\text{C3}]\right)$, where d_i is the slippage in days; the module flags $s_i > 3$. The score is used for the ranking-based metrics reported in Section 7.4. C1–C3 are implemented as `v_anomalies_detection` over the monthly fact table and served read-only at `/api/analitica/anomalias`, so the rule evaluated in Section 7.4 is the rule an operator runs.

3.6 A Note on Identifiers

This article uses English identifiers for tables, columns and views; the published implementation uses Spanish ones, as it was built with and for a Spanish-speaking municipal office. The correspondence is mechanical —`fact_audit_events` is `fact_eventos_auditoria`, `v_delayed_works` is `v_obras_retraso`, `dim_work` is `dim_obra`, and so on— and the repository README tabulates every pair, so that a reader inspecting the code can locate each object referenced here. This mapping is a prerequisite for the reproducibility claims of Section 7.

4 Data Warehouse Design

The warehouse adopts a dimensional star model [2] with two fact tables and ten dimensions, shown in Figure 2 with the SCD type of each.

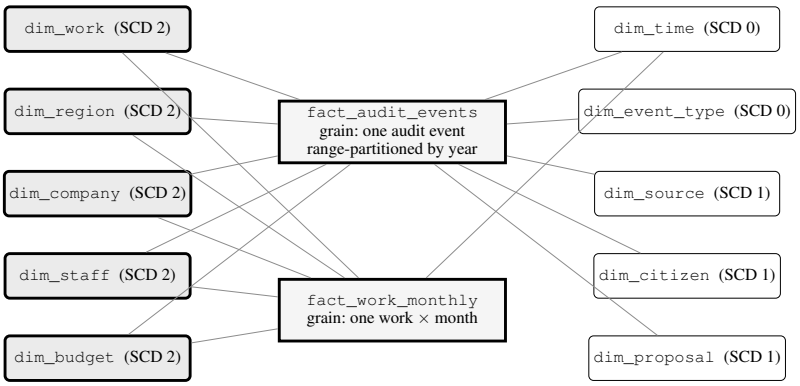

Fig. 2. Dimensional star schema, redrawn for legibility. Two fact tables (`fact_audit_events`, range-partitioned by year, and `fact_work_monthly`) are connected to the ten dimensions through surrogate keys. The five Slowly Changing Dimensions of Type 2 (shaded, left) are those that carry the historical traceability of the works. The monthly fact table is conformed only with the five SCD 2 dimensions and the time dimension.

4.1 Fact Tables and Grain

We state the grain of each fact table explicitly, since it determines every subsequent aggregation and is a prerequisite for reproducing the design.

`fact_audit_events` has a grain of *one row per audit event*: a single, atomic, immutable action performed on a work by an identified actor at an identified instant. It records events classified into 17 types (work creation, work modification, state change, progress report, cost record, budget record, budget modification, image upload, permit issue, hand-over record, contractor selection, funding assignment, supervisor assignment, role change, proposal registration, vote cast and session start) and is partitioned by year. The seeded scenario writes 8,934 events directly; the synchronisation triggers emit more as reports, costs, images and votes are registered, so a populated instance holds about 28,000. Each row carries foreign keys to the ten dimensions, financial metrics, progress metrics, and audit flags.

fact_work_monthly has a grain of *one row per work per calendar month*. It is a periodic snapshot table: for each work and month it stores the total budget, the accumulated cost, the remaining balance, the percentage executed, the physical progress, the number of reports and images registered, and a boolean delay flag with the associated slippage in days. Because the grain is a snapshot rather than a transaction, aggregating cost across months requires taking the last snapshot of the period rather than a sum; the analytical views encapsulate this rule so that consumers cannot get it wrong.

4.2 Slowly Changing Dimensions

The SCD 2 dimensions [2] (five in total: works, regions, companies, staff, and budget) maintain history through validity columns (`effective_date`, `expiration_date`, `is_current`), which makes it possible to reconstruct the exact state of any work at an arbitrary point in time.

Three representative use cases motivate this choice, none answerable with a Type 1 dimension. In a *budget-modification audit*, `dim_budget` closes the current version and opens a new one, so an auditor can ask which amount was in force on the date a payment was authorised, not merely the amount in force today. Under *contractor reassignment*, `dim_company` preserves both versions, so progress recorded before the change stays attributed to the original contractor —a precondition for any performance analysis by contractor. Under *territorial reorganisation*, where communities are merged or renamed, `dim_region` versioning prevents a historical investment series from being silently rewritten.

The remaining five do not version. `dim_time` and `dim_event_type` are fixed catalogues (Type 0). `dim_source` is Type 1: it carries running totals of works financed and amount assigned, which are overwritten as they change. `dim_citizen` and `dim_proposal` are Type 1 because only their current state is analytically meaningful and, since they hold names and national identity codes, retaining superseded versions would create a personal-data liability with no analytical return. The distribution is five Type 2, three Type 1 and two Type 0. `scripts/verificar_scd.py` checks the schema against this classification, so the two cannot drift apart.

4.3 Analytical Views and Refresh Strategy

Six analytical views derived from the fact tables support analytical consumption: (1) **v_delayed_works** (works with a schedule slip and days of delay); (2) **v_audit_alerts** (alerts classified as HIGH_AMOUNT, CANCELLED_WORK, BUDGET_CHANGE, LATE_EVENT and REVIEW); (3) **v_work_traceability** (full SCD 2 version history of a work); (4) **v_citizen_participation** (votes and proposals by region); (5) **v_budget_executed** (accumulated budget execution by funding source); and (6) **v_anomalies_detection** (the C1–C3 criteria of Section 3.5 evaluated per work and month, with one column per criterion).

These are standard SQL views rather than materialized ones. The choice is deliberate and determines the refresh strategy. At the volumes involved —of the order of 10^4 fact rows for 64,844 inhabitants— the views execute in tens of milliseconds over the partitioned fact table, so materialisation would buy little while introducing a staleness window; and since alerts must be actionable the moment a supervisor uploads a report, a live view is semantically preferable, with no refresh job to schedule

or to fail. Should the deployment scale to several municipalities, the migration path is to materialise the aggregation-heavy `v_budget_executed` and `v_citizen_participation` with `REFRESH MATERIALIZED VIEW CONCURRENTLY` after each nightly load, keeping the two alert-bearing views live.

5 Object Storage Layer: Evidence Storage

The object layer uses Cloudflare R2, a service compatible with the Amazon S3 API, selected for its zero egress cost and reduced global latency. In its current state it functions as an object repository for binary evidence (images and PDFs) linked to the warehouse, and not as a queryable analytical lake: analytical queries are resolved in the Data Warehouse, not over R2. Adopting an open table format [9] that enables direct queries over the lake is proposed as future work (Section 8). The path design follows the *hive-partitioned* convention [3] of Equation (1).

Ingestion is direct from the web form: the supervisor attaches images or PDFs when creating a progress report; the API computes the object key, uploads it through the S3 SDK, records the metadata in the warehouse, and returns the public URL. This is not an atomic transaction across the two stores: object storage and PostgreSQL cannot enlist in one. The implementation compensates —on a database failure it issues a best-effort delete of the objects just uploaded— so a crash between the two steps can still leave an orphaned object. Reconciling them would need a sweep against the metadata table, which is not implemented.

6 Geospatial Interface Prototype

The geospatial display is an *implicit* graph at the interface level —the relationships are derived from the foreign keys of the dimensional model— and not a formal knowledge graph with an ontology or an RDF triple store; publishing the schema as a linked vocabulary is addressed in future work.

Figure 3 shows the main interface, with markers color-coded by the state of the work and the technical-card popup that displays physical and financial progress indicators when a node is selected. It is part of the released artefact, reachable from the navigation bar of every page. Each marker sits at the real coordinates of its community, resolved from OpenStreetMap, with a deterministic offset of at most 150 m so that two works in the same community do not overlap. The community is therefore real; the exact site within it is not surveyed.

6.1 Management and Visualization Components

- **Marker layer:** each active work is represented by a marker color-coded by state —green (completed), amber (in progress), red (delayed)— derived from the work's latest progress report.
- **Technical card (popup):** selecting a work displays its file number, name, community, total budget, assigned company, date range, a physical-progress bar, and an expandable section for images from the object layer.
- **Analysis layer:** highlights delayed works and indicates the days of slippage (criterion C2 of Section 3.5); the dual physical/financial percentage detects works with high budget execution and low physical progress (criterion C3), a potential indicator of irregularities.

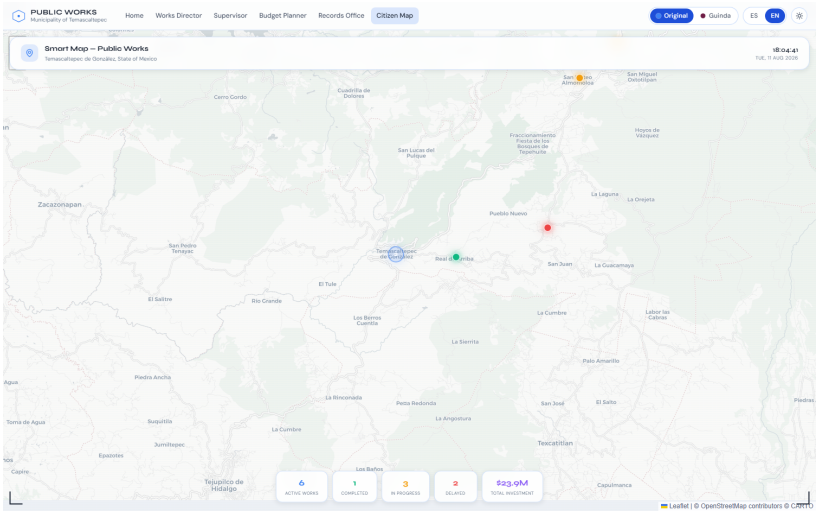

Fig. 3. The published geospatial viewer, in light mode. Markers are color-coded by state —green completed, amber in progress, red delayed— over the six works of the demonstration dataset, with the indicator panel below. Live at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/mapa/?modo=claro&idioma=en; captured in a 1500×938 viewport. The viewer opens on a fixed centre and zoom over the municipality, so the window size is what decides how much surrounding territory is in frame.

7 Results and Evaluation

7.1 Scope of the Evaluation

Measured results and projections are kept strictly apart in what follows. We label every figure in this section as **measured** (obtained by executing published code or auditing the published artefact, and reproducible by a third party: Sections 7.2, 7.3 and 7.4) or **projected** (an expected operational outcome not yet observed in any deployment: Section 7.5). No number in this article is a measurement taken on real municipal data.

Three synthetic populations appear here and must not be conflated. The *published site* holds six works in the visitor's browser tab and supports only interface claims and the audit of Section 7.3. The *seeded warehouse* —1,247 works, 55 communities, $127.4 M, 29,928 monthly fact rows— supports the structural claims and the latencies of Section 7.2. The *detection dataset*, a separate simulation of about 2,580 work–period records per seed, supports Table 2 and nothing else.

7.2 End-to-End Latency (Measured)

Latency is measured against the reference stack in `docker/` —PostgreSQL and the Flask API in containers, loaded with the seeded population— and is reproducible with `scripts/medir_rendimiento.py`.

Environment: Windows 10 host, Docker 29.6, PostgreSQL 16.14 in a container with 12 CPUs, Flask behind gunicorn with two synchronous workers, 1,247 works, 14,885 progress reports and 29,928 monthly fact rows.

Table 1. Latency measured against the reference stack, with `scripts/medir_rendimiento.py --repeticiones 100`. API rows are 100 serial requests over the seeded population of 1,247 works; view rows are the server execution time reported by `EXPLAIN ANALYZE` over 25 runs; throughput is 100 requests across ten concurrent clients.

Operation	Mean	p95
`/api/public/obras` (753 kB response)	566 ms	610 ms
`/api/public/resumen`	544 ms	589 ms
`/api/public/regiones`	10.9 ms	17.7 ms
`/api/analitica/anomalias`	18.3 ms	33.4 ms
`v_delayed_works`	2.0 ms	2.2 ms
`v_anomalies_detection`	23.8 ms	24.5 ms
`v_budget_executed`	5.2 ms	5.4 ms
Insert firing the SCD 2 trigger	0.1 ms	0.1 ms
Sustained throughput, 10 clients		3.4 req/s

The result does not flatter the design uniformly. The warehouse answers in single-digit milliseconds: the delayed-works view runs in 2 ms and an insert firing the SCD 2 synchroniser costs 0.1 ms, so neither the star schema nor the versioning triggers are the limit at this scale. The delivery layer is where the time goes. `/api/public/obras` takes 566 ms because it serialises the entire portfolio in one unpaginated response, and sustained throughput is 3.4 req/s, which is what two synchronous workers and a 566 ms response give. Pagination and more workers are the remedy, and neither touches the dimensional design. Map load, gallery rendering and object-store latency are not reported: measuring them needs a deployed map backend and an R2 bucket the artefact does not include, and a gap is preferable to an uncheckable number.

The seeded warehouse holds 1,247 works across 55 communities, 8,934 audit events, 3,421 photographic pieces of evidence, $127.4 million pesos in budget, 2,156 citizen proposals and 8,723 votes cast.

7.3 Delivery of the Published Frontend (Measured)

The static artefact is permanently online, so its delivery can be audited by anyone against a public URL. A Lighthouse audit of its two entry points, taken as the median of three runs each on 11 August 2026 (Lighthouse 12.8.2, headless Chrome 151, emulated mobile with simulated throttling, tag `v1.7.3-icokg2026`), gives the landing page 85 for performance and the map 72, with 100 for accessibility, 96 and 93 for best practices and 90 and 100 for SEO; both pages leave the main thread idle and shift no layout, and first contentful paint is 3.2 s and 3.6 s. What separates them is delivery weight: render-blocking fonts and a CDN animation library on one side, a 533 kB bundle on the other. Reaching 100 on accessibility meant fixing what the audit reported —a brand link with no accessible name below 720 px, an attribution painted below 4.5:1, two icon-only buttons, and map markers that are `role="button"` elements with no text— rather than restating the score.

This audit measures the delivery of a static page. It executes no Flask, no PostgreSQL and no warehouse query, so it is evidence about the front end alone; the architecture is exercised in Section 7.2.

7.4 Evaluation of the Delay/Alert Detection Module (Measured)

Generator. Anomalies are now latent states of a simulated execution process, and what the detector sees are their observable consequences. A work is drawn with a contract price, a planned duration and a productivity parameter, and is then executed month by month under seasonal and remoteness effects; payment follows progress with a mobilisation advance, a lag and noise. Five latent causes are injected: a price inflated at award, an unproductive contractor, payments running ahead of execution, a work that stops, and a work contracted and paid but never built. Their magnitudes are drawn from distributions that *overlap the normal range*, so some anomalies are undetectable in the recorded data and some healthy works look irregular. That overlap is the condition for the evaluation to be informative.

Protocol. Ten seeds and four latent prevalences (5, 10, 15 and 20%); the main configuration is 15%, which yields about 2,580 work–period records per seed of which 17.2% are anomalous. We report the mean over seeds with a 95% confidence interval. The baseline is an Isolation Forest [12] (100 estimators, contamination set to the latent prevalence) over five normalised features. Because comparing a rule at a fixed threshold against a baseline at its default threshold compares two operating points rather than two detectors, the baseline is also evaluated at a *matched alert budget*: the k highest-scoring records, with k the number of alerts the rule raises.

Table 2. Detection performance at 15% latent prevalence. Mean over ten seeds with a 95% confidence interval; anomalies are latent process states, not detector predicates. Reproducible with the published scripts.

Method	Precision	Recall	F1	ROC-AUC	PR-AUC
Threshold rule (this work)	$.462 \pm .056$	$.351 \pm .031$	$.397 \pm .038$	$.630 \pm .022$	$.310 \pm .052$
Isolation Forest (default)	$.369 \pm .053$	$.321 \pm .031$	$.342 \pm .037$	$.650 \pm .029$	$.342 \pm .052$
Isolation Forest (matched budget)	$.387 \pm .054$	$.294 \pm .033$	$.333 \pm .038$	$.650 \pm .029$	$.342 \pm .052$

Results. The rule alerts on about a third of the anomalies and is right slightly less than half the time. It does not dominate the baseline: Isolation Forest ranks better on both AUCs at every prevalence tested, so as a scoring function the rule is the poorer of the two. Its advantage is the operating point —at the same alert budget it is more precise (.462 against .387) and recovers more (.351 against .294)— which matters because an oversight office reviews a fixed number of files per period, not a ranking. Both improve with prevalence (the rule from .217 to .557 precision between 5 and 20%).

Where it fails. Per-cause recall is uneven in a way that is diagnostic. The rule recovers unproductive contractors at $.548 \pm .059$, ghost works at $.474 \pm .024$ and stopped works at $.386 \pm .012$, but overpricing at only $.093 \pm .051$ and payments running ahead of execution at $.061 \pm .034$. The reason is structural, not a matter of tuning. C1 compares an absolute amount against the mean *of its period*, so it flags large works rather than overpriced ones: a school inflated by 30% is still cheaper than an honest road. And no criterion examines the *gap* between physical and financial progress unless the physical figure falls below 30%, so a work paid steadily ahead of its execution passes unremarked if it eventually finishes. The rule is a schedule-failure filter with a narrow ghost-work check attached, not a cost-anomaly detector.

7.5 Illustrative Analytical Capability (Projected)

The warehouse answers questions a spreadsheet cannot: investment per inhabitant by community against the development index, budget concentration across funding sources, and participation rates by region. Over the seeded population these queries return coherent, plausible figures, which shows the analytical pipeline runs end to end. They are properties of the generator and nothing else: no claim about territorial inequality, budget concentration or savings in Temascaltepec is made or implied, and none can be until the model is populated with municipal records.

8 Conclusions and Future Work

8.1 Conclusions

This work presented a pragmatic architecture centered on a dimensional Data Warehouse for the geospatial monitoring of municipal public works, complementing the analytical store with S3-compatible object storage for unstructured evidence under a single REST API. Returning to the research questions: RQ1 is answered affirmatively and, in this version, empirically: ten dimensions, five of them SCD 2, two fact tables and six views on commodity managed services, measured over the reference stack of Section 7.2, where the analytical layer answers in single-digit milliseconds and the limit lies in the delivery layer rather than in the dimensional design. The caveat is that the measurement is of a reference deployment under synthetic load, not of a production installation serving a municipality. RQ2 is answered in Section 4: an event-grain transaction fact table plus a work-month snapshot table, with SCD 2 restricted to the five audit-relevant entities, suffices to serve delay detection, budget alerting and participation analytics from one schema. RQ3 is answered in Section 7.4: the rule attains 0.46 precision at 0.35 recall against latent causes and is out-ranked by an unsupervised baseline, with its failures concentrated in cost anomalies, whose criterion is mis-specified.

The proposal shows that it is possible to build a transparency system with analytical capability (OLAP views, SCD 2 history), georeferenced photographic evidence, and citizen participation, without relying on high-cost enterprise platforms.

8.2 Limitations

(1) There is no hosted dynamic deployment: permanence rests on the static artefact, and the API is distributed to be run by whoever needs it rather than kept online by us. (2) The participation module uses a simple voting scheme that may not adequately represent minorities. (3) The quality of the analysis depends on the quality, coverage, and freshness of the data entered by field supervisors. (4) All analytical results come from synthetic data —the warehouse population from a seeded Faker generator, the detection results from a process simulation over seeds 42 to 51— because privacy constraints preclude publishing municipal records; nothing here has been validated against real ones. (5) The architecture is, in its current state, *data-warehouse-centric*: the object store holds binary evidence but is not queried analytically. (6) The public map endpoints read the operational schema rather than the warehouse views, so the two read paths can diverge; unifying them is pending. (7) Access control is lightweight —the staff role travels in a request header and the citizen session token, though HMAC-signed, has no expiry— which is adequate for a prototype whose analytical surface is public by design, but must be hardened before handling non-public data.

8.3 Future Work

(0) Evolve the object layer toward a full lakehouse by adopting an open table format [9] (Apache Iceberg, or Parquet with transactional metadata) queryable directly over R2 through a lightweight engine such as DuckDB or Trino, without introducing Apache Spark; (1) a computer-vision inference pipeline to automatically classify work progress from the stored photographs; (2) publishing the schema as an OWL vocabulary to interoperate with Linked Data, and exposing an OC4IDS-conformant [10] projection of the warehouse so that the municipality's data becomes comparable with other CoST-aligned jurisdictions; (3) quadratic voting to improve the representativeness of minority proposals; (4) time-series models to predict completion dates; (5) recomputing the cost-anomaly criterion per work type, following the failure analysis of Section 7.4, and re-evaluating against the same ground truth; and (6) validating the architecture in additional municipalities, with a pre/post measurement of report-preparation time and information-request volume.

8.4 Availability

The warehouse DDL, SCD 2 triggers, seeded generators, load-testing scripts and the evaluation script are archived at https://github.com/gabrielhuav/PublicMunicipalWorks_DWH. A static demonstration is published through GitHub Pages at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/. It preserves the interface of the original prototype —four role workspaces, the citizen-participation module and the map— replacing remote authentication and writes with synthetic in-browser behaviour, and adds Spanish/English switching and two colour themes in light and dark mode; it makes no request to any backend service. Access is open by construction: the credentials are printed on the landing page and the form accepts any value, and every screen is served from a synthetic dataset held in the visitor's own tab, so no input can affect another visitor. The Flask implementation in `backend/` is the operational reference, with the same `DEMO-` accounts constrained to read-only. It is runnable rather than archived: `docker/` raises PostgreSQL and the API with the seeded warehouse, so an agency wanting to operate the system starts from a working stack, and the figures above were verified against that container. Table 2 is reproduced by running `eval_deteccion_v2.py` from `scripts/evaluacion/`, which drives the generator over seeds 42–51 and the four prevalences; the latencies of Section 7.2 by `scripts/medir_rendimiento.py` against the container, and the declared dimension types by `scripts/verificar_scd.py` against the schema. The population figures are constants in `scripts/generate_synthetic_data.py`, met by construction; its `--verificar` flag checks them without a database.

Declaration on the Use of Generative AI

The authors used a generative AI assistant, under their direction, for language editing, LaTeX formatting, drafting text subsequently revised and approved by the authors, and writing the auxiliary scripts distributed with the artefact, and generating the artefact's illustrative work images, each labelled as such in the file. The research questions, the dimensional design, the ETL, the data collection, the experiments and the interpretation of the results are the authors' own work.

Disclosure of Interests

The authors declare that they have no conflicts of interest relevant to the content of this article.

References

1. Zaharia, M., et al.: Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In: Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR 2021). Online (2021)
2. Kimball, R., Ross, M.: The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd edn. Wiley, Indianapolis (2013)
3. Palkar, S., Thusoo, A., Boncz, P.A.: Building a geospatial lakehouse, part 1. Databricks Engineering Blog (2021). https://www.databricks.com/blog/2021/12/17/building-a-geospatial-lakehouse-part-1.html. Last accessed 15 Jan 2026
4. Hogan, A., et al.: Knowledge graphs. ACM Comput. Surv. **54**(4), Article 71 (2021). https://doi.org/10.1145/3447772
5. Janowicz, K., et al.: Know, know where, KnowWhereGraph: a densely connected, cross-domain knowledge graph and geo-enrichment service stack for applications in environmental intelligence. AI Mag. **43**(1), 30–39 (2022). https://doi.org/10.1002/aaai.12043
6. Espinoza-Arias, P., Fernández-Ruiz, M.J., Morlán-Plo, V., Notivol-Bezares, R., Corcho, O.: The Zaragoza's knowledge graph: open data to harness the city knowledge. Information **11**(3), 129 (2020). https://doi.org/10.3390/info11030129
7. Lourenço, R.P.: An analysis of open government portals: a perspective of transparency for accountability. Gov. Inf. Q. **32**(3), 323–332 (2015). https://doi.org/10.1016/j.giq.2015.05.006
8. Rajabi, E., Midha, R., de Souza, J.F.: Constructing a knowledge graph for open government data: the case of Nova Scotia disease datasets. J. Biomed. Semant. **14**, 4 (2023). https://doi.org/10.1186/s13326-023-00284-w
9. Armbrust, M., et al.: Delta Lake: high-performance ACID table storage over cloud object stores. Proc. VLDB Endow. **13**(12), 3411–3424 (2020). https://doi.org/10.14778/3415478.3415560
10. Open Contracting Partnership, CoST — the Infrastructure Transparency Initiative: Open Contracting for Infrastructure Data Standard (OC4IDS). https://standard.open-contracting.org/infrastructure/latest/en/. Last accessed 1 Aug 2026
11. Villa Vargas, J.M., Hurtado Avilés, G., Climent Hernández, J.A.: Cuando México tiembla: la historia contada por los datos. AZCATL Rev. Divulg. Cienc. Ing. Innov. **4**(6), 28–33 (2026). https://doi.org/10.24275/AZC2026E1004
12. Liu, F.T., Ting, K.M., Zhou, Z.-H.: Isolation forest. In: Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008), pp. 413–422. IEEE Computer Society, Pisa (2008). https://doi.org/10.1109/ICDM.2008.17
13. Rivest, S., Bédard, Y., Proulx, M.-J., Nadeau, M., Hubert, F., Pastor, J.: SOLAP technology: merging business intelligence with geospatial technology for interactive spatio-temporal exploration and analysis of data. ISPRS J. Photogramm. Remote Sens. **60**(1), 17–33 (2005). https://doi.org/10.1016/j.isprsjprs.2005.10.002
14. Faisal, S., Sarwar, M.: Handling slowly changing dimensions in data warehouses. J. Syst. Softw. **94**, 151–160 (2014). https://doi.org/10.1016/j.jss.2014.03.072
15. Ambituuni, A.: Infrastructure transparency initiative and anticorruption in public infrastructure projects: a principal–agent perspective to the antecedents that enable (or hinder) the effectiveness of transparency. Eng. Constr. Archit. Manag. **33**(7), 5934–5957 (2026). https://doi.org/10.1108/ECAM-02-2025-0273
16. Fazekas, M., Kocsis, G.: Uncovering high-level corruption: cross-national objective corruption risk indicators using public procurement data. Br. J. Polit. Sci. **50**(1), 155–164 (2020). https://doi.org/10.1017/S0007123417000461